

The AllowPartiallyTrustedCallersAttribute (APTCA) attribute enables access to fully trusted code from partially trusted code in signed assemblies. When you apply the APTCA attribute to an assembly, partially trusted callers can access that assembly for the life of the assembly, which can compromise security.

What to do if your app fails this test

Don't use the APTCA attribute on strong named assemblies unless your project requires it and the risks are well understood. In cases where it's required, make sure that all APIs are protected with appropriate code access security demands. APTCA has no effect when the assembly is a part of a Universal Windows Platform (UWP) app.

Remarks

This test is performed only on managed code (C#, .NET, etc.).

/SafeSEH Exception Handling Protection

Windows App Certification Kit error message: SafeSEHCheck Test failed

An exception handler runs when the app encounters an exceptional condition, such as a divide-by-zero error. Because the address of the exception handler is stored on the stack when a function is called, it could be vulnerable to a buffer overflow attacker if some malicious software were to overwrite the stack.

What to do if your app fails this test

Enable the /SAFESEH option in the linker command when you build your app. This option is on by default in the Release configurations of Visual Studio. Verify this option is enabled in the build instructions for all executable modules in your app.

Remarks

The test is not performed on 64-bit binaries or Arm chipset binaries because they don't store exception handler addresses on the stack.

Data Execution Prevention

Windows App Certification Kit error message: NXCheck Test failed

This test verifies that an app doesn't run code that is stored in a data segment.

What to do if your app fails this test

Enable the `/NXCOMPAT` option in the linker command when you build your app. This option is on by default in linker versions that support Data Execution Prevention (DEP).

Remarks

We recommend that you test your apps on a DEP-capable CPU and fix any failures you find that result from DEP.

Address Space Layout Randomization

Windows App Certification Kit error message: DBCheck Test failed

Address Space Layout Randomization (ASLR) loads executable images into unpredictable locations in memory, which makes it harder for malicious software that expects a program to be loaded at a certain virtual address to operate predictably. Your app and all components that your app uses must support ASLR.

What to do if your app fails this test

Enable the `/DYNAMICBASE` option in the linker command when you build your app. Verify that all modules that your app uses also use this linker option.

Remarks

Normally, ASLR doesn't affect performance. But in some scenarios there is a slight performance improvement on 32-bit systems. It is possible that performance could degrade in a highly congested system that have many images loaded in many different memory locations.

This test is performed only on apps written in unmanaged languages, such as by using C or C++.

Read/Write Shared PE Section

Windows App Certification Kit error message: SharedSectionsCheck Test failed.

Binary files with writable sections that are marked as shared are a security threat. Don't build apps with shared writable sections unless necessary. Use [CreateFileMapping](#) or [MapViewOfFile](#) to create a properly secured shared memory object.

What to do if your app fails this test

Remove any shared sections from the app and create shared memory objects by calling [CreateFileMapping](#) or [MapViewOfFile](#) with the proper security attributes and then

rebuild your app.

Remarks

This test is performed only on apps written in unmanaged languages, such as by using C or C++.

AppContainerCheck

Windows App Certification Kit error message: AppContainerCheck Test failed.

The AppContainerCheck verifies that the **appcontainer** bit in the portable executable (PE) header of an executable binary is set. Apps must have the **appcontainer** bit set on all .exe files and all unmanaged DLLs to execute properly.

What to do if your app fails this test

If a native executable file fails the test, make sure that you used the latest compiler and linker to build the file and that you use the */appcontainer* flag on the linker.

If a managed executable fails the test, make sure that you used the latest compiler and linker, such as Microsoft Visual Studio, to build the UWP app.

Remarks

This test is performed on all .exe files and on unmanaged DLLs.

ExecutableImportsCheck

Windows App Certification Kit error message: ExecutableImportsCheck Test failed.

A portable executable (PE) image fails this test if its import table has been placed in an executable code section. This can occur if you enabled .rdata merging for the PE image by setting the */merge* flag of the Visual C++ linker as */merge:.rdata=.text*.

What to do if your app fails this test

Don't merge the import table into an executable code section. Make sure that the */merge* flag of the Visual C++ linker is not set to merge the ".rdata" section into a code section.

Remarks

This test is performed on all binary code except purely managed assemblies.

WXCheck

Windows App Certification Kit error message: WXCheck Test failed.

The check helps to ensure that a binary does not have any pages that are mapped as writable and executable. This can occur if the binary has a writable and executable section or if the binary's *SectionAlignment* is less than *PAGE-SIZE*.

What to do if your app fails this test

Make sure that the binary does not have a writeable or executable section and that the binary's *SectionAlignment* value is at least equal to its *PAGE-SIZE*.

Remarks

This test is performed on all .exe files and on native, unmanaged DLLs.

An executable may have a writable and executable section if it has been built with Edit and Continue enabled (/ZI). Disabling Edit and Continue will cause the invalid section to not be present.

PAGE-SIZE is the default *SectionAlignment* for executables.

Private Code Signing

Tests for the existence of private code signing binaries within the app package.

Background

Private code signing files should be kept private as they may be used for malicious purposes in the event they are compromised.

Test details

Tests for files within the app package that have an extension of .pfx or .snk that would indicate that private signing keys were included.

Corrective actions

Remove any private code signing keys (for example, .pfx and .snk files) from the package.

Supported API test

Test the app for the use of any non-compliant APIs.

Background

Apps must use the APIs for UWP apps (Windows Runtime or supported Win32 APIs) to be certified for the Microsoft Store. This test also identifies situations where a managed binary takes a dependency on a function outside of the approved profile.

Test details

- Verifies that each binary within the app package doesn't have a dependency on a Win32 API that is not supported for UWP app development by checking the import address table of the binary.
- Verifies that each managed binary within the app package doesn't have a dependency on a function outside of the approved profile.

Corrective actions

Make sure that the app was compiled as a release build and not a debug build.

Note The debug build of an app will fail this test even if the app uses only [APIs for UWP apps](#).

Review the error messages to identify the API the app uses that is not an [API for UWP apps](#).

Note C++ apps that are built in a debug configuration will fail this test even if the configuration only uses APIs from the Windows SDK for UWP apps. See, [Alternatives to Windows APIs in UWP apps](#) for more info.

Performance tests

The app must respond quickly to user interaction and system commands in order to present a fast and fluid user experience.

The characteristics of the computer on which the test is performed can influence the test results. The performance test thresholds for app certification are set such that low-power computers meet the customer's expectation of a fast and fluid experience. To

determine your app's performance, we recommend that you test on a low-power computer, such as an Intel Atom processor-based computer with a screen resolution of 1366x768 (or higher) and a rotational hard drive (as opposed to a solid-state hard drive).

Bytecode generation

As a performance optimization to accelerate JavaScript execution time, JavaScript files ending in the .js extension generate bytecode when the app is deployed. This significantly improves startup and ongoing execution times for JavaScript operations.

Test Details

Checks the app deployment to verify that all .js files have been converted to bytecode.

Corrective Action

If this test fails, consider the following when addressing the issue:

- Verify that event logging is enabled.
- Verify that all JavaScript files are syntactically valid.
- Confirm that all previous versions of the app are uninstalled.
- Exclude identified files from the app package.

Optimized binding references

When using bindings, WinJS.Binding.optimizeBindingReferences should be set to true in order to optimize memory usage.

Test Details

Verify the value of WinJS.Binding.optimizeBindingReferences.

Corrective Action

Set WinJS.Binding.optimizeBindingReferences to **true** in the app JavaScript.

App manifest resources test

App resources validation

The app might not install if the strings or images declared in your app's manifest are incorrect. If the app does install with these errors, your app's logo or other images used by your app might not display correctly.

Test Details

Inspects the resources defined in the app manifest to make sure they are present and valid.

Corrective Action

Use the following table as guidance.

 Expand table

Error message	Comments
The image {image name} defines both Scale and TargetSize qualifiers; you can define only one qualifier at a time.	You can customize images for different resolutions. In the actual message, {image name} contains the name of the image with the error. Make sure that each image defines either Scale or TargetSize as the qualifier.
The image {image name} failed the size restrictions.	Ensure that all the app images adhere to the proper size restrictions. In the actual message, {image name} contains the name of the image with the error.
The image {image name} is missing from the package.	A required image is missing. In the actual message, {image name} contains the name of the image that is missing.
The image {image name} is not a valid image file.	Ensure that all the app images adhere to the proper file format type restrictions. In the actual message, {image name} contains the name of the image that is not valid.
The image "BadgeLogo" has an ABGR value {value} at position (x, y) that is not valid. The pixel must be white (##FFFFFF) or transparent (00#####)	The badge logo is an image that appears next to the badge notification to identify the app on the lock screen. This image must be monochromatic (it can contain only white and transparent pixels).

	In the actual message, {value} contains the color value in the image that is not valid.
The image "BadgeLogo" has an ABGR value {value} at position (x, y) that is not valid for a high-contrast white image. The pixel must be (##2A2A2A) or darker, or transparent (00#####).	The badge logo is an image that appears next to the badge notification to identify the app on the lock screen. Because the badge logo appears on a white background when in high-contrast white, it must be a dark version of the normal badge logo. In high-contrast white, the badge logo can only contain pixels that are darker than (##2A2A2A) or transparent. In the actual message, {value} contains the color value in the image that is not valid.
The image must define at least one variant without a TargetSize qualifier. It must define a Scale qualifier or leave Scale and TargetSize unspecified, which defaults to Scale-100.	For more info, see Responsive design 101 for UWP apps and Guidelines for app resources .
The package is missing a "resources.pri" file.	If you have localizable content in your app manifest, make sure that your app's package includes a valid resources.pri file.
The "resources.pri" file must contain a resource map with a name that matches the package name {package full name}	You can get this error if the manifest changed and the name of the resource map in resources.pri no longer matches the package name in the manifest. In the actual message, {package full name} contains the package name that resources.pri must contain. To fix this, you need to rebuild resources.pri and the easiest way to do that is by rebuilding the app's package.
The "resources.pri" file must not have AutoMerge enabled.	MakePRI.exe supports an option called AutoMerge . The default value of AutoMerge is off . When enabled, AutoMerge merges an app's language pack resources into a single resources.pri at runtime. We don't recommend this for apps that you intend to distribute through the Microsoft Store. The resources.pri of an app that is distributed through the Microsoft Store must be in the root of the app's package and contain all the language references that the app supports.
The string {string} failed the max length restriction of {number} characters.	Refer to the App package requirements. In the actual message, {string} is replaced by the string with the error and {number} contains the maximum length.
The string {string} must not have leading/trailing whitespace.	The schema for the elements in the app manifest don't allow leading or trailing white space characters. In the actual message, {string} is replaced by the string with the error.

	Make sure that none of the localized values of the manifest fields in resources.pri have leading or trailing white space characters.
The string must be non-empty (greater than zero in length)	For more info, see App package requirements .
There is no default resource specified in the "resources.pri" file.	For more info, see Guidelines for app resources . In the default build configuration, Visual Studio only includes scale-200 image resources in the app package when generating bundles, putting other resources in the resource package. Make sure you either include scale-200 image resources or configure your project to include the resources you have.
There is no resource value specified in the "resources.pri" file.	Make sure that the app manifest has valid resources defined in resources.pri.
The image file {filename} must be smaller than 204800 bytes.**	Reduce the size of the indicated images.
The {filename} file must not contain a reverse map section.**	While the reverse map is generated during Visual Studio 'F5 debugging' when calling into makepri.exe, it can be removed by running makepri.exe without the /m parameter when generating a pri file.
** Indicates that a test was added in the Windows App Certification Kit 3.3 for Windows 8.1 and is only applicable when using the that version of the kit or later.	

Branding validation

UWP apps are expected to be complete and fully functional. Apps using the default images (from templates or SDK samples) present a poor user experience and cannot be easily identified in the store catalog.

Test Details

The test will validate if the images used by the app are not default images either from SDK samples or from Visual Studio.

Corrective actions

Replace default images with something more distinct and representative of your app.

Debug configuration test

Test the app to make sure it is not a debug build.

Background

To be certified for the Microsoft Store, apps must not be compiled for debug and they must not reference debug versions of an executable file. In addition, you must build your code as optimized for your app to pass this test.

Test details

Test the app to make sure it is not a debug build and is not linked to any debug frameworks.

Corrective actions

- Build the app as a release build before you submit it to the Microsoft Store.
- Make sure that you have the correct version of .NET framework installed.
- Make sure the app isn't linking to debug versions of a framework and that it is building with a release version. If this app contains .NET components, make sure that you have installed the correct version of the .NET framework.

File encoding test

UTF-8 file encoding

Background

HTML, CSS, and JavaScript files must be encoded in UTF-8 form with a corresponding byte-order mark (BOM) to benefit from bytecode caching and avoid certain runtime error conditions.

Test details

Test the contents of app packages to make sure that they use the correct file encoding.

Corrective Action

Open the affected file and select **Save As** from the **File** menu in Visual Studio. Select the drop-down control next to the **Save** button and select **Save with Encoding**. From the **Advanced** save options dialog, choose the Unicode (UTF-8 with signature) option and click **OK**.

Direct3D feature level test

Direct3D feature level support

Tests Microsoft Direct3D apps to ensure that they won't crash on devices with older graphics hardware.

Background

Microsoft Store requires all applications using Direct3D to render properly or fail gracefully on feature level 9-1 graphics cards.

Because users can change the graphics hardware in their device after the app is installed, if you choose a minimum feature level higher than 9-1, your app must detect at launch whether or not the current hardware meets the minimum requirements. If the minimum requirements are not met, the app must display a message to the user detailing the Direct3D requirements. Also, if an app is downloaded on a device with which it is not compatible, it should detect that at launch and display a message to the customer detailing the requirements.

Test Details

The test will validate if the apps render accurately on feature level 9-1.

Corrective Action

Ensure that your app renders correctly on Direct3D feature level 9-1, even if you expect it to run at a higher feature level. See [Developing for different Direct3D feature levels](#) for more info.

Direct3D Trim after suspend

Note This test only applies to UWP apps developed for Windows 8.1 and later.

Background

If the app does not call [Trim](#) on its Direct3D device, the app will not release memory allocated for its earlier 3D work. This increases the risk of apps being terminated due to system memory pressure.

Test Details

Checks apps for compliance with d3d requirements and ensures that apps are calling a new [Trim](#) API upon their Suspend callback.

Corrective Action

The app should call the [Trim](#) API on its [IDXGIDevice3](#) interface anytime it is about to be suspended.

App Capabilities test

Special use capabilities

Background

Special use capabilities are intended for very specific scenarios. Only company accounts are allowed to use these capabilities.

Test Details

Validate if the app is declaring any of the below capabilities:

- EnterpriseAuthentication
- SharedUserCertificates
- DocumentsLibrary

If any of these capabilities are declared, the test will display a warning to the user.

Corrective Actions

Consider removing the special use capability if your app doesn't require it. Additionally, use of these capabilities are subject to additional on-boarding policy review.

Windows Runtime metadata validation

Background

Ensures that the components that ship in an app conform to the Windows Runtime type system.

Test Details

Verifies that the `.winmd` files in the package conform to Windows Runtime rules.

Corrective Actions

- **ExclusiveTo attribute test:** Ensure that Windows Runtime classes don't implement interfaces that are marked as ExclusiveTo another class.
- **Type location test:** Ensure that the metadata for all Windows Runtime types is located in the winmd file that has the longest namespace-matching name in the app package.
- **Type name case-sensitivity test:** Ensure that all Windows Runtime types have unique, case-insensitive names within your app package. Also ensure that no UWP type name is also used as a namespace name within your app package.
- **Type name correctness test:** Ensure there are no Windows Runtime types in the global namespace or in the Windows top-level namespace.
- **General metadata correctness test:** Ensure that the compiler you are using to generate your types is up to date with the Windows Runtime specifications.
- **Properties test:** ensure that all properties on a Windows Runtime class have a get method (set methods are optional). Ensure that the type of the get method return value matches the type of the set method input parameter, for all properties on Windows Runtime types.

Package Sanity tests

Platform appropriate files test

Apps that install mixed binaries may crash or not run correctly depending upon the user's processor architecture.

Background

This test validates the binaries in an app package for architecture conflicts. An app package should not include binaries that can't be used on the processor architecture specified in the manifest. Including unsupported binaries can lead to your app crashing or an unnecessary increase in the app package size.

Test Details

Validates that each file's "bitness" in the PE header is appropriate when cross-referenced with the app package processor architecture declaration

Corrective Action

Follow these guidelines to ensure that your app package only contains files supported by the architecture specified in the app manifest:

- If the Target Processor Architecture for your app is Neutral processor Type, the app package cannot contain x86, x64, or Arm binary or image type files.
- If the Target Processor Architecture for your app is x86 processor type, the app package must only contain x86 binary or image type files. If the package contains x64 or Arm binary or image types, it will fail the test.
- If the Target Processor Architecture for your app is x64 processor type, the app package must contain x64 binary or image type files. Note that in this case the package can also include x86 files, but the primary app experience should utilize the x64 binary.

However, if the package contains Arm binary or image type files, or only contains x86 binaries or image type files, it will fail the test.

- If the Target Processor Architecture for your app is Arm processor type, the app package must only contain Arm binary or image type files. If the package contains x64 or x86 binary or image type files, it will fail the test.

Supported Directory Structure test

Validates that applications are not creating subdirectories as part of installation that are longer than MAX-PATH.

Background

OS components (including Trident, WWAHost, etc.) are internally limited to MAX-PATH for file system paths and will not work correctly for longer paths.

Test Details

Verifies that no path within the app install directory exceeds MAX-PATH.

Corrective Action

Use a shorter directory structure, and or file name.

Resource Usage test

WinJS Background Task test

WinJS background task test ensures that JavaScript apps have the proper close statements so apps don't consume battery.

Background

Apps that have JavaScript background tasks need to call Close() as the last statement in their background task. Apps that do not do this could keep the system from returning to connected standby mode and result in draining the battery.

Test Details

If the app does not have a background task file specified in the manifest, the test will pass. Otherwise the test will parse the JavaScript background task file that is specified in the app package, and look for a Close() statement. If found, the test will pass; otherwise the test will fail.

Corrective Action

Update the background JavaScript code to call Close() correctly.

Related topics

- [Windows Desktop Bridge app tests](#)
- [Microsoft Store Policies](#)

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Windows Desktop Bridge app tests

Article • 10/24/2022

Desktop Bridge apps (see [Building an MSIX package from your code](#)) are Windows desktop apps converted to Universal Windows Platform (UWP) apps by using the [Desktop Bridge](#). After conversion, the Windows desktop application is packaged, serviced, and deployed in the form of a UWP app package (a .appx or .appxbundle) targeting Windows 10 Desktop.

Required versus optional tests

Optional tests for Windows Desktop Bridge apps are informational only and will not be used to evaluate your app during Microsoft Store onboarding. We recommend investigating these test results to produce better quality apps. The overall pass/fail criteria for store onboarding is determined by the required tests and not by these optional tests.

Current optional tests

1. Digitally signed file test

Background

This test verifies that all portable executable (PE) files contain a valid signature. The presence of digitally signed files allows users to know that the software is genuine.

Test details

The test scans all of the portable executable files in the package and checks their headers for a signature. All the PE files are recommended to be digitally signed. A warning will be generated if any of the PE files is not signed.

Corrective actions

Having digitally signed files is always recommended. For more information, see [Introduction to Code Signing](#).

2. File association verbs

Background

This test scans the package registry to check if any file association verbs are registered.

Test details

Converted desktop apps can be enhanced with a wide range of Windows Runtime APIs. This test checks that the UWP binaries in the app don't call non-Windows Runtime APIs. UWP binaries have the **AppContainer** flag set.

Corrective actions

See [Desktop to UWP Bridge: App extensions](#) for an explanation of these extensions and how to use them properly.

3. Debug configuration test

This test verifies that the .msix or .appx is not a debug build.

Background

To be certified for the Microsoft Store, apps must not be compiled for debug and they must not reference debug versions of an executable file. In addition, you must build your code as optimized for your app to pass this test.

Test details

Test the app to make sure it is not a debug build and is not linked to any debug frameworks.

Corrective actions

- Build the app as a release build before you submit it to the Microsoft Store.
- Make sure that you have the correct version of .NET framework installed.
- Make sure the app isn't linking to debug versions of a framework and that it is building with a release version. If this app contains .NET components, make sure that you have installed the correct version of the .NET framework.

4. Package sanity test

4.1 Archive files usage

Background

This test helps you build better Desktop Bridge Apps to run on [Windows 10 S](#) machines.

Test details

This test checks for all executable files inside of archived files or self-extracting content. As executable files contained within this type of content are not signed during

onboarding to Windows store, the app might not run as expected on Windows 10 S systems.

Corrective actions

- Consider evaluating the files flagged by the test to determine if there is impact to your app running in a Windows 10 S environment.
- If your app would be affected, remove the executable files from the archived files and do not use self-extracting archives to place executable files on disk. This should prevent the loss of app functionality.

4.2 Blocked executables

Background

This test helps you build better Desktop Bridge Apps to run on [Windows 10 S](#) machines.

Test details

This test checks whether the app is attempting to launch executable files, which is restricted on Windows 10 S systems. Apps that rely on this capability may not run as expected on Windows 10 S systems.

Corrective actions

- Identify which of the flagged entries from the test represent a call to launch an executable file that is not part of your app, and remove those calls.
- If the flagged file(s) is part of your application, you may ignore the warning.

Current required tests

1. App Capabilities test (special use capabilities)

Background

Special use capabilities are intended for very specific scenarios. Only company accounts are allowed to use these capabilities.

Test details

Validate if the app is declaring any of the below capabilities:

- EnterpriseAuthentication
- SharedUserCertificates
- DocumentsLibrary

If any of these capabilities are declared, the test will display a warning to the user.

Corrective actions

Consider removing the special use capability if your app doesn't require it. Additionally, use of these capabilities is subject to additional onboarding policy review.

2. App manifest resources tests

2.1 App resources validation

Your app might not install properly if the strings or images declared in the app's manifest are incorrect. If the app does install with these errors, your app's logo or other images may not display correctly.

Test details

Inspects the resources defined in the app manifest to make sure they are present and valid.

Corrective action

Use the following table as guide.

 Expand table

Error message	Comments
The image {image name} defines both Scale and TargetSize qualifiers; you can define only one qualifier at a time.	You can customize images for different resolutions. In the actual message, {image name} contains the name of the image with the error. Make sure that each image defines either Scale or TargetSize as the qualifier.
The image {image name} failed the size restrictions.	Ensure that all the app images adhere to the proper size restrictions. In the actual message, {image name} contains the name of the image with the error.
The image {image name} is missing from the package.	A required image is missing. In the actual message, {image name} contains the name of the image that is missing.
The image {image name} is not a valid image file.	Ensure that all the app images adhere to the proper file format type restrictions. In the actual message, {image name} contains the name of the image that is not valid.
The image "BadgeLogo" has an ABGR value {value} at position (x, y) that is not valid. The pixel must be white (FFFFFF) or transparent (00####).	The badge logo is an image that appears next to the badge notification to identify the app on the lock screen. This image must be monochromatic (it can contain only white and transparent pixels). In the actual message, {value} contains the color value in the image that is not valid.

Error message	Comments
The image "BadgeLogo" has an ABGR value {value} at position (x, y) that is not valid for a high-contrast white image. The pixel must be (##2A2A2A) or darker, or transparent (00#####).	The badge logo is an image that appears next to the badge notification to identify the app on the lock screen. Because the badge logo appears on a white background when in high-contrast white, it must be a dark version of the normal badge logo. In high-contrast white, the badge logo can only contain pixels that are darker than (##2A2A2A) or transparent. In the actual message, {value} contains the color value in the image that is not valid.
The image must define at least one variant without a TargetSize qualifier. It must define a Scale qualifier or leave Scale and TargetSize unspecified, which defaults to Scale-100.	For more info, see the guides on responsive design and app resources .
The package is missing a "resources.pri" file.	If you have localizable content in your app manifest, make sure that your app's package includes a valid resources.pri file.
The "resources.pri" file must contain a resource map with a name that matches the package name {package full name}	You can get this error if the manifest changed and the name of the resource map in resources.pri no longer matches the package name in the manifest. In the actual message, {package full name} contains the package name that resources.pri must contain. To fix this, you need to rebuild resources.pri and the easiest way to do that is by rebuilding the app's package.
The "resources.pri" file must not have AutoMerge enabled.	MakePRI.exe supports an option called AutoMerge. The default value of AutoMerge is off. When enabled, AutoMerge merges an app's language pack resources into a single resources.pri at runtime. We don't recommend this for apps that you intend to distribute through the Microsoft Store. The resources.pri of an app that is distributed through the Microsoft Store must be in the root of the app's package and contain all the language references that the app supports.
The string {string} failed the max length restriction of {number} characters.	Refer to the App package requirements . In the actual message, {string} is replaced by the string with the error and {number} contains the maximum length.
The string {string} must not have leading/trailing whitespace.	The schema for the elements in the app manifest don't allow leading or trailing white space characters. In the actual message, {string} is replaced by the string with the error. Make sure that none of the localized values of the manifest fields in resources.pri have leading or trailing white space characters.
The string must be non-empty (greater than zero in length)	For more info, see App package requirements .

Error message	Comments
There is no default resource specified in the "resources.pri" file.	For more info, see the guide on app resources . In the default build configuration, Visual Studio only includes scale-200 image resources in the app package when generating bundles, putting other resources in the resource package. Make sure you either include scale-200 image resources or configure your project to include the resources you have.
There is no resource value specified in the "resources.pri" file.	Make sure that the app manifest has valid resources defined in resources.pri.
The image file {filename} must be smaller than 204800 bytes.	Reduce the size of the indicated images.
The {filename} file must not contain a reverse map section.	While the reverse map is generated during Visual Studio 'F5 debugging' when calling into makepri.exe, it can be removed by running makepri.exe without the /m parameter when generating a pri file.

2.2 Branding validation

Background

Desktop Bridge Apps are expected to be complete and fully functional. Apps using the default images (from templates or SDK samples) present a poor user experience and cannot be easily identified in the store catalog.

Test details

The test will validate if the images used by the app are not default images either from SDK samples or from Visual Studio.

Corrective actions

Replace default images with something more distinct and representative of your app.

3. Package compliance tests

3.1 App Manifest

Tests the contents of app manifest to make sure its contents are correct.

Background

Apps must have a correctly formatted app manifest.

Test details

Examines the app manifest to verify the contents are correct as described in the [App package requirements](#). The following checks are done in this test:

- **File extensions and protocols**

Your app may declare the file types that it can be associated with. A declaration of a large number of uncommon file types makes for a poorer user experience. This test limits the number of file extensions that an app can be associated with.

- **Framework dependency rule**

This test enforces the requirement that apps declare appropriate dependencies on the UWP. If there is an inappropriate dependency, this test will fail. If there is a mismatch between the OS version the app targets and the framework dependencies made, the test will fail. The test also fails if the app refers to any "preview" versions of the framework dlls.

- **Inter-process communication (IPC) verification**

This test enforces the requirement that Desktop Bridge apps do not communicate outside of the app container to desktop components. Inter-process communication is intended for side-loaded apps only. Apps that specify the [ActivatableClassAttribute](#) with name equal to `DesktopApplicationPath` will fail this test.

Corrective action

Review the app's manifest against the requirements described in the [App package requirements](#).

3.2 Application Count

This test verifies that an app package (.appx, app bundle) contains one application.

Background

This test is implemented as per Store policy.

Test details

This test verifies that the total number of .appx packages in the bundle is less than 512, and that there is only one "main" package in the bundle. It also verifies that the revision number of the bundle version is set to 0.

Corrective actions

Ensure the app package and bundle meet requirements stated in **Test details**.

3.3 Registry Checks

Background

This test checks if the application installs or updates any new services or drivers.

Test details

The test looks inside the registry.dat file for updates to specific registry locations that indicate a new service or driver being registered. If the app is attempting to install a driver or service, the test will fail.

Corrective actions

Review the failures and remove the services or drivers in question if they are unnecessary. If the app depends on these, you will need to revise the app if you want to onboard to the Store.

4. Platform appropriate files test

Apps that install mixed binaries may crash or not run correctly depending on the user's processor architecture.

Background

This test scans the binaries in an app package for architecture conflicts. An app package should not include binaries that can't be used on the processor architecture specified in the manifest. Including unsupported binaries can lead to your app crashing or an unnecessary increase in the app package size.

Test details

Validates that each file's "bitness" in the portable executable header is appropriate when cross-referenced with the app package processor architecture declaration.

Corrective actions

Follow these guidelines to ensure that your app package only contains files supported by the architecture specified in the app manifest:

- If the Target Processor Architecture for your app is Neutral Processor Type, the app package cannot contain x86, x64, or Arm binary or image type files.
- If the Target Processor Architecture for your app is x86 processor type, the app package must only contain x86 binary or image type files. If the package contains x64 or Arm binary or image types, it will fail the test.
- If the Target Processor Architecture for your app is x64 processor type, the app package must contain x64 binary or image type files. Note that in this case the package can also include x86 files, but the primary app experience should utilize the x64 binary. If the package contains Arm binary or image type files, or *only* contains x86 binaries or image type files, it will fail the test.

- If the Target Processor Architecture for your app is Arm processor type, the app package must only contain Arm binary or image type files. If the package contains x64 or x86 binary or image type files, it will fail the test.

5. Supported API test

Checks the app for the use of any non-compliant APIs.

Background

Desktop Bridge apps can leverage some legacy Win32 APIs along with modern APIs (UWP components). This test identifies managed binaries that use unsupported APIs.

Test details

This test checks all the UWP components in the app:

- Verifies that each managed binary within the app package doesn't have a dependency on a Win32 API that is not supported for UWP app development by checking the import address table of the binary.
- Verifies that each managed binary within the app package doesn't have a dependency on a function outside of the approved profile.

Corrective actions

This can be corrected by ensuring that the app was compiled as a release build and not as a debug build.

ⓘ Note

The debug build of an app will fail this test even if the app uses only [APIs for UWP apps](#). Review the error messages to identify the API present that is not an allowed API for UWP apps.

ⓘ Note

C++ apps that are built in a debug configuration will fail this test even if the configuration only uses APIs from the Windows SDK for UWP apps. See [Alternatives to Windows APIs in UWP apps](#) for more information.

6. User account control (UAC) test

Background

Ensures that the app is not requesting user account control at runtime.

Test details

An app cannot request admin elevation or UIAccess per Microsoft Store policy. Elevated security permissions are not supported.

Corrective actions

Apps must run as an interactive user. See [UI Automation Security Overview](#) for details.

7. Windows Runtime metadata validation

Background

Ensures that the components that ship in an app conform to the UWP type system.

Test details

This test throws a number of flags related to proper type usage.

Corrective actions

- **ExclusiveTo attribute**
Ensure that UWP classes don't implement interfaces that are marked as ExclusiveTo another class
- **General Metadata correctness**
Ensure that the compiler you are using to generate your types is up-to-date with the UWP specifications.
- **Properties**
Ensure that all properties in a UWP class have a `get` method (`set` methods are optional). For all properties, ensure that the type returned by the `get` method matches the type of the `set` method input parameter.
- **Type location**
Ensure that the metadata for all UWP types is located in the .winmd file that has the longest namespace-matching name in the app package.
- **Type name case-sensitivity**
Ensure that all UWP types have unique, case-insensitive names within your app package. Also ensure that no UWP type name is also used as a namespace name within your app package.
- **Type name correctness**
Ensure there are no UWP types in the global namespace or in the Windows top-level namespace.

8. Windows security features tests

Changing the default Windows security protections can put customers at increased risk.

8.1 Banned File Analyzer

Background

Certain files have been updated with important security, reliability or other improvements. Windows Desktop Bridge apps must contain the latest versions these files, as outdated versions present a risk. The Windows App Certification Kit blocks these files to ensure that all apps use the current version.

Test details

The Banned File Check in the Windows App Certification Kit currently checks for the following files:

- *Bing.Maps.JavaScript\js\veapicore.js*

This check commonly fails when an app is using a "Release Preview" version of the file instead of the latest official release.

Corrective actions

To correct this, use the latest version of the [Bing Maps SDK](#) for UWP apps.

8.2 Private Code Signing

Tests for the existence of private code signing binaries within the app package.

Background

Private code signing files should be kept private as they may be used for malicious purposes in the event they are compromised.

Test details

Checks for files within the app package that have an extension of .pfx or .snk that would indicate that private signing keys are included.

Corrective actions

Remove any private code signing keys (such as .pfx and .snk files) from the package.

Related topics

- [Microsoft Store Policies](#)

Feedback

Was this page helpful?

 Yes

 No

Performance

Article • 10/20/2022

Users expect their apps to remain responsive, to feel natural, and not to drain their battery. Technically, performance is a non-functional requirement but treating performance as a feature will help you deliver on your users' expectations. Specifying goals, and measuring, are key factors. Determine what your performance-critical scenarios are; define what good performance mean. Then measure early and often enough throughout the lifecycle of your project to be confident you'll hit your goals.. This section shows you how to organize your performance workflow, fix animation glitches and frame rate problems, and tune your startup time, page navigation time, and memory usage.

If you haven't done so already, a step that we've seen result in significant performance improvements is just porting your app to target Windows 10. Several XAML optimizations (for example, `{x:Bind}`) are only available in Windows 10 apps. See [Porting apps to Windows 10](#) and the //build/ session [Moving to the Universal Windows Platform](#).

Topic	Description
Planning for performance	Users expect their apps to remain responsive, to feel natural, and not to drain their battery. Technically, performance is a non-functional requirement but treating performance as a feature will help you deliver on your users' expectations. Specifying goals, and measuring, are key factors. Determine what your performance-critical scenarios are; define what good performance mean. Then measure early and often enough throughout the lifecycle of your project to be confident you'll hit your goals.
Optimize background activity	Create UWP apps that work with the system to use background tasks in a battery-efficient way.
ListView and GridView UI optimization	Improve GridView performance and startup time through UI virtualization, element reduction, and progressive updating of items.
ListView and GridView data virtualization	Improve GridView performance and startup time through data virtualization.
Improve garbage collection performance	Universal Windows Platform (UWP) apps written in C# and Visual Basic get automatic memory management from the .NET garbage collector. This section summarizes the behavior and performance best practices for the .NET garbage collector in UWP apps.

Topic	Description
Keep the UI thread responsive	Users expect an app to remain responsive while it does computation, regardless of the type of machine. This means different things for different apps. For some, this translates to providing more realistic physics, loading data from disk or the web faster, quickly presenting complex scenes and navigating between pages, finding directions in a snap, or rapidly processing data. Regardless of the type of computation, users want their app to act on their input and eliminate instances where it appears unresponsive while it "thinks".
Optimize your XAML markup	Parsing XAML markup to construct objects in memory is time-consuming for a complex UI. Here are some things you can do to improve XAML markup parse and load time and memory efficiency for your app.
Optimize your XAML layout	Layout can be an expensive part of a XAML app—both in CPU usage and memory overhead. Here are some simple steps you can take to improve the layout performance of your XAML app.
MVVM and language performance tips	This topic discusses some performance considerations related to your choice of software design patterns, and programming language.
Best practices for your app's startup performance	Create UWP apps with optimal startup times by improving the way you handle launch and activation.
Optimize animations, media, and images	Create Universal Windows Platform (UWP) apps with smooth animations, high frame rate, and high-performance media capture and playback.
Optimize suspend/resume	Create UWP apps that streamline their use of the process lifetime system to resume efficiently after suspension or termination.
Optimize file access	Create UWP apps that access the file system efficiently, avoiding performance issues due to disk latency and memory/CPU cycles.
Windows Runtime components and optimizing interop	Create UWP apps that use UWP Components and interop between native and managed types while avoiding interop performance issues.
Tools for profiling and performance	Microsoft provides several tools to help you improve the performance of your UWP app.

Planning for performance

Article • 10/20/2022

Users expect their apps to remain responsive, to feel natural, and not to drain their battery. Technically, performance is a non-functional requirement but treating performance as a feature will help you deliver on your users' expectations. Specifying goals, and measuring, are key factors. Determine what your performance-critical scenarios are; define what good performance mean. Then measure early and often enough throughout the lifecycle of your project to be confident you'll hit your goals.

Specifying goals

The user experience is a basic way to define good performance. An app's startup time can influence a user's perception of its performance. A user might consider an app launch time of less than one second to be excellent, less than 5 seconds to be good, and greater than 5 seconds to be poor.

Other metrics have a less obvious impact on user experience, for example memory. The chances of an app being terminated while either suspended or inactive rise with the amount of memory used by the active app. It's a general rule that high memory usage degrades the experience for all apps on the system, so having a goal on memory consumption is reasonable. Take into consideration the rough size of your app as perceived by users: small, medium, or large. Expectations around performance will correlate to this perception. For example, you might want a small app that doesn't use a lot of media to consume less than 100MB of memory.

It's better to set an initial goal, and then revise it later, than not to have a goal at all. Your app's performance goals should be specific and measurable and they should fall into three categories: how long it takes users, or the app, to complete tasks (time); the rate and continuity with which the app redraws itself in response to user interaction (fluidity); and how well the app conserves system resources, including battery power (efficiency).

Time

Think of the acceptable ranges of elapsed time (*interaction classes*) it takes for users to complete their tasks in your app. For each interaction class assign a label, a perceived user sentiment, and ideal and maximum durations. Here are some suggestions.

Interaction class label	User perception	Ideal	Maximum	Examples
Fast	Minimally noticeable delay	100 milliseconds	200 milliseconds	Bring up the app bar; press a button (first response)
Typical	Quick, but not fast	300 milliseconds	500 milliseconds	Resize; semantic zoom
Responsive	Not quick, but feels responsive	500 milliseconds	1 second	Navigate to a different page; resume the app from a suspended state
Launch	Competitive experience	1 second	3 seconds	Launch the app for the first time or after it has been previously terminated
Continuous	No longer feels responsive	500 milliseconds	5 seconds	Download a file from the Internet
Captive	Long; user could switch away	500 milliseconds	10 seconds	Install multiple apps from the Store

You can now assign interaction classes to your app's performance scenarios. You can assign the app's point-in-time reference, a portion of the user experience, and an interaction class to each scenario. Here are some suggestions for an example food and dining app.

Scenario	Time point	User experience	Interaction class
Navigate to recipe page	First response	Page transition animation started	Fast (100-200 milliseconds)
	Responsive	Ingredients list loaded; no images	Responsive (500 milliseconds - 1 second)
	Visible complete	All content loaded; images shown	Continuous (500 milliseconds - 5 seconds)
Search for recipe	First response	Search button clicked	Fast (100 - 200 milliseconds)
	Visible complete	List of local recipe titles shown	Typical (300 - 500 milliseconds)

If you're displaying live content then also consider content freshness goals. Is the goal to refresh content every few seconds? Or is refreshing content every few minutes, every

few hours, or even once a day an acceptable user experience?

With your goals specified, you are now better able to test, analyze, and optimize your app.

Fluidity

Specific measurable fluidity goals for your app might include:

- No screen redraw stops-and-starts (glitches).
- Animations render at 60 frames per second (FPS).
- When a user pans/scrolls, the app presents 3-6 pages of content per second.

Efficiency

Specific measurable efficiency goals for your app might include:

- For your app's process, CPU percentage is at or below N and memory usage in MB is at or below M at all times.
- When the app is inactive, N and M are zero for your app's process.
- Your app can be used actively for X hours on battery power; when your app is inactive, the device retains its charge for Y hours.

Design your app for performance

You can now use your performance goals to influence your app's design. Using the example food and dining app, after the user navigates to the recipe page, you might choose to [update items incrementally](#) so that the recipe's name is rendered first, displaying the ingredients is deferred, and displaying images is deferred further. This maintains responsiveness and a fluid UI while panning/scrolling, with the full fidelity rendering taking place after the interaction slows to a pace that allow the UI thread to catch up. Here are some other aspects to consider.

UI

- Maximize parse and load time and memory efficiency for each page of your app's UI (especially the initial page) by [optimizing your XAML markup](#). In a nutshell, defer loading UI and code until it's needed.
- For [ListView](#) and [GridView](#), make all the items the same size and use as many [ListView and GridView optimization techniques](#) as you can.

- Declare UI in the form of markup, which the framework can load and re-use in chunks, rather than constructing it imperatively in code.
- Delay creating UI elements until the user needs them. See the [x:Load](#) attribute.
- Prefer theme transitions and animations to storyboarded animations. For more info, see [Animations overview](#). Remember that storyboarded animations require constant updates to the screen, and keep the CPU and graphics pipeline active. To preserve the battery, don't have animations running if the user is not interacting with the app.
- Images you load should be loaded at a size that is appropriate for the view in which you are presenting it, using the [GetThumbnailAsync](#) method.

CPU, memory, and power

- Schedule lower-priority work to run on lower-priority threads and/or cores. See [Asynchronous programming](#), the [Dispatcher](#) property, and the [CoreDispatcher](#) class.
- Minimize your app's memory footprint by releasing expensive resources (such as media) on suspend.
- Minimize your code's working set.
- Avoid memory leaks by unregistering event handlers and dereferencing UI elements whenever possible.
- For the sake of the battery, be conservative with how often you poll for data, query a sensor, or schedule work on the CPU when it is idle.

Data access

- If possible, prefetch content. For automatic prefetching, see the [ContentPrefetcher](#) class. For manual prefetching, see the [Windows.ApplicationModel.Background](#) namespace and the [MaintenanceTrigger](#) class.
- If possible, cache content that's expensive to access. See the [LocalFolder](#) and [LocalSettings](#) properties.
- For cache misses, show a placeholder UI as quickly as possible that indicates that the app is still loading content. Transition from placeholder to live content in a way that is not jarring to the user. For example, don't change the position of content under the user's finger or mouse pointer as the app loads live content.

App launch and resume

- Defer the app's splash screen, and don't extend the app's splash screen unless necessary. For details, see [Creating a fast and fluid app launch experience](#) [↗](#) and [Display a splash screen for more time](#).
- Disable animations that occur immediately after the splash screen is dismissed, as these will only lead to a perception of delay in app launch time.

Adaptive UI, and orientation

- Use the [VisualStateManager](#) class.
- Complete only required work immediately, deferring intensive app work until later—your app has between 200 and 800 milliseconds to complete work before the user sees your app's UI in a cropped state.

With your performance-related designs in place, you can start coding your app.

Instrument for performance

As you code, add code that logs messages and events at certain points while your app runs. Later, when you're testing your app, you can use profiling tools such as Windows Performance Recorder and Windows Performance Analyzer (both are included in the [Windows Performance Toolkit](#)) to create and view a report about your app's performance. In this report, you can look for these messages and events to help you more easily analyze the report's results.

The Universal Windows Platform (UWP) provides logging APIs, backed by [Event Tracing for Windows \(ETW\)](#), that together offer a rich event logging and tracing solution. The APIs, which are part of the [Windows.Foundation.Diagnostics](#) namespace, include the [FileLoggingSession](#), [LoggingActivity](#), [LoggingChannel](#), and [LoggingSession](#) classes.

To log a message in the report at a specific point while the app is running, create a [LoggingChannel](#) object, and then call the object's [LogMessage](#) method, like this.

C#

```
// using Windows.Foundation.Diagnostics;
// ...

LoggingChannel myLoggingChannel = new LoggingChannel("MyLoggingChannel");

myLoggingChannel.LogMessage(LoggingLevel.Information, "Here' s my logged
message.");

// ...
```

To log start and stop events in the report over a period of time while the app is running, create a [LoggingActivity](#) object, and then call the object's [LoggingActivity](#) constructor, like this.

C#

```
// using Windows.Foundation.Diagnostics;
// ...

LoggingActivity myLoggingActivity;

// myLoggingChannel is defined and initialized in the previous code example.
using (myLoggingActivity = new LoggingActivity("MyLoggingActivity"),
myLoggingChannel))
{
    // After this logging activity starts, a start event is logged.

    // Add code here to do something of interest.

}
// After this logging activity ends, an end event is logged.

// ...
```

Also see the [Logging sample](#).

With your app instrumented, you can test and measure your app's performance.

Test and measure against performance goals

Part of your performance plan is to define the points during development where you'll measure performance. This serves different purposes depending on whether you're measuring during prototyping, development, or deployment. Measuring performance during the early stages of prototyping can be tremendously valuable, so we recommend that you do so as soon as you have code that does meaningful work. Early measurements give you a good idea of where the important costs are in your app, and inform design decisions. This results in high performing and scaling apps. It's generally costlier to change designs later than earlier. Measuring performance late in the product cycle can result in last-minute hacks and poor performance.

Use these techniques and tools to test how your app stacks up against your original performance goals.

- Test against a wide variety of hardware configurations including all-in-one and desktop PCs, laptops, ultrabooks, and tablets and other mobile devices.
- Test against a wide variety of screen sizes. While wider screen sizes can show much more content, bringing in all of that extra content can negatively impact performance.
- Eliminate as many testing variables as you can.
 - Turn off background apps on the testing device. To do this, in Windows, select **Settings** from the Start menu > **Personalization** > **Lock screen**. Select each active app and select **None**.

- Compile your app to native code by building it in release configuration before deploying it to the testing device.
- To ensure that automatic maintenance does not affect the performance of the testing device, trigger it manually and wait for it to complete. In Windows, in the Start menu search for **Security and Maintenance**. In the **Maintenance** area, under **Automatic Maintenance**, select **Start maintenance** and wait for the status to change from **Maintenance in progress**.
- Run the app multiple times to help eliminate random testing variables and help ensure consistent measurements.
- Test for reduced power availability. Your users' device might have significantly less power than your development machine. Windows was designed with low-power devices, such as mobile devices, in mind. Apps that run on the platform should ensure they perform well on these devices. As a heuristic, expect that a low power device runs at about a quarter the speed of a desktop computer, and set your goals accordingly.
- Use a combination of tools like Microsoft Visual Studio and Windows Performance Analyzer to measure app performance. Visual Studio is designed to provide app-focused analysis, such as source code linking. Windows Performance Analyzer is designed to provide system-focused analysis, such as providing system info, info about touch manipulation events, and info about disk input/output (I/O) and graphics processing unit (GPU) cost. Both tools provide trace capture and export, and can reopen shared and post-mortem traces.
- Before you submit your app to the Store for certification, be sure to incorporate into your test plans the performance-related test cases as described in the "Performance tests" section of [Windows App Certification Kit tests](#) and in the "Performance and stability" section of [UWP app test cases](#).

For more info, see these resources and profiling tools.

- [Windows Performance Analyzer](#)
- [Windows Performance Toolkit](#)
- [Analyze performance using Visual Studio diagnostic tools](#)
- The //build/ session XAML Performance
- The //build/ session New XAML Tools in Visual Studio 2015

Respond to the performance test results

After you analyze your performance test results, determine if any changes are needed, for example:

- Should you change any of your app design decisions, or optimize your code?

- Should you add, remove, or change any of the instrumentation in the code?
- Should you revise any of your performance goals?

If any changes are needed, make them and then go back to instrumenting or testing and repeat.

Optimizing

Optimize only the performance-critical code paths in your app: those where most time is spent. Profiling will tell you which. Often, there is a trade-off between creating software that follows good design practices and writing code that performs at the highest optimization. It is generally better to prioritize developer productivity and good software design in areas where performance is not a concern.

Optimize background activity

Article • 10/20/2022

Universal Windows apps should perform consistently well across all device families. On battery-powered devices, power consumption is a critical factor in the user's overall experience with your app. All-day battery life is a desirable feature to every user, but it requires efficiency from all of the software installed on the device, including your own.

Background task behavior is arguably the most important factor in the total energy cost of an app. A background task is any program activity that has been registered with the system to run without the app being open. See [Create and register an out-of-process background task](#) for more information.

Background activity permissions

On desktop and mobile devices running Windows 10, version 1607 or later, users can view their "Battery usage by app" in the Battery section of the Settings app. Here they will see a list of apps and the percentage of battery life that each app has consumed (out of the amount of battery life that has been used since the last charge). For UWP apps on this list, users can select the app to open up controls related to background activity.



Background permissions on mobile

On mobile devices, users will see a list of radio buttons that specify the background task permission setting for that app. Background activity can be set to "Always allowed," "Never allowed," or "Managed by Windows," which means the app's background activity is regulated by the system according to a number of factors.



Battery usage by app

Time: 24 Hours ▾

Showing: Apps with usage ▾



OneNote

39%



In use: 38% Background: 1%

- ☐ Always allowed in background
- ☒ Managed by Windows
- ☐ Never allowed in background



Word Mobile

23%

Managed by Windows



Mail and Calendar

7%

Managed by Windows



Background permissions on desktop

On desktop devices, the "Managed by Windows" setting is presented as a toggle switch, set to **On** by default. If the user switches to **Off**, they are presented with a checkbox with which they can manually define the background activity permissions. When the box is checked, the app will be allowed to run background tasks at all times. When the box is unchecked, the background activity will be disabled.



Battery usage by app

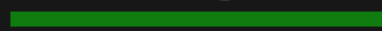
Time: 24 Hours ▾

Showing: Apps with usage ▾



Microsoft Edge

54%



In use: 39% Background: 15%

Managed By Windows



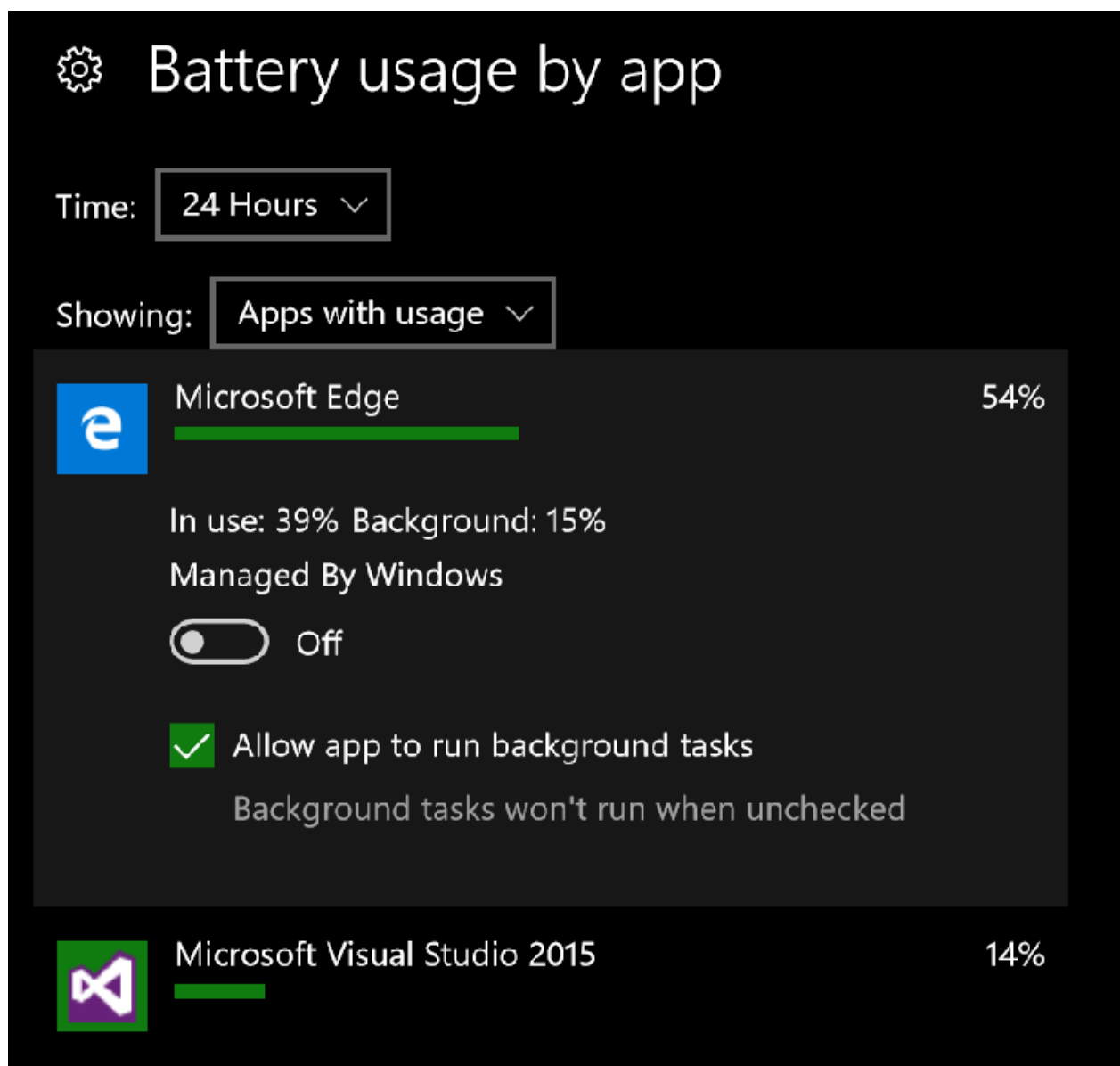
On



Microsoft Visual Studio 2015

14%





In your app, you can use the [BackgroundAccessStatus](#) enum value returned by a call to the [BackgroundExecutionManager.RequestAccessAsync\(\)](#) method to determine its current background activity permission setting.

All this is to say that if your app doesn't implement responsible background activity management, the user may deny background permissions to your app altogether, which is not desirable for either party. If your app has been denied permission to run in the background but requires background activity to complete an action for the user, you can notify the user and point them to the Settings app. This can be accomplished by [Launching the Settings App](#) to the Background Apps or Battery Usage Details page.

Work with the Battery Saver feature

Battery Saver is a system-level feature that users can configure in Settings. It cuts off all background activity of all apps when the battery level drops below a user-defined

threshold, *except* for the background activity of apps that have been set to "Always allowed."

Check the status of Battery Saver mode from within your app by referencing the [PowerManager.EnergySaverStatus](#) property. It is an enum value: either **EnergySaverStatus.Disabled**, **EnergySaverStatus.Off** or **EnergySaverStatus.On**. If your app requires background activity and is not set to "Always allowed" then it should handle **EnergySaverStatus.On** by notifying the user that the given background task(s) will not run until Battery Saver is off. While background activity management is the primary purpose of the Battery Saver feature, your app can make additional adjustments to further conserve energy when Battery Saver is on. In the case where Battery Saver is on, your app could reduce its use of animations, stop location polling, or delay syncs and backups.

Further optimize background tasks

The following are additional steps you can take when registering your background tasks to make them more battery-aware.

Use a maintenance trigger

A [MaintenanceTrigger](#) object can be used instead of a [SystemTrigger](#) object to determine when a background task starts. Tasks that use maintenance triggers will only run when the device is connected to AC power, and they are allowed to run for longer. See [Use a maintenance trigger](#) for instructions.

Use the BackgroundWorkCostNotHigh system condition type

System conditions must be met in order for background tasks to run (see [Set conditions for running a background task](#) for more). The background work cost is a measurement that denotes the *relative* energy impact of running the background task. A task running when the device is plugged into AC power would be marked as **low** (little/no impact on battery). A task running when the device is on battery power with the screen off is marked as **high** because there is presumably little program activity running on the device at the time, so the background task would have a greater relative cost. A task running when the device is on battery power with the screen *on* is marked as **medium**, because there is presumably already some program activity running, and the background task would add a bit more to the energy cost. The

BackgroundWorkCostNotHigh system condition simply delays your task's ability to run until either the screen is on or the device is connected to AC power.

Test battery efficiency

Be sure to test your app on real devices for any high-power-consumption scenarios. It's a good idea to test your app on many different devices, with Battery Saver on and off, and in environments of varying network strength.

Related topics

- [Create and register an out-of-process background task](#)
- [Planning for performance](#)

ListView and GridView UI optimization

Article • 01/04/2023

Note For more details, see the [//build/ session Dramatically Increase Performance when Users Interact with Large Amounts of Data in GridView and ListView](#).

Improve [ListView](#) and [GridView](#) performance and startup time through UI virtualization, element reduction, and progressive updating of items. For data virtualization techniques, see [ListView and GridView data virtualization](#).

Two key factors in collection performance

Manipulating collections is a common scenario. A photo viewer has collections of photos, a reader has collections of articles/books/stories, and a shopping app has collections of products. This topic shows what you can do to make your app efficient at manipulating collections.

There are two key factors in performance when it comes to collections: one is the time spent by the UI thread creating items; the other is the memory used by both the raw data set and the UI elements used to render that data.

For smooth panning/scrolling, it's vital that the UI thread do an efficient and smart job of instantiating, data-binding, and laying out items.

UI virtualization

UI virtualization is the most important improvement you can make. This means that UI elements representing the items are created on demand. For an items control bound to a 1000-item collection, it would be a waste of resources to create the UI for all the items at the same time, because they can't all be displayed at the same time. [ListView](#) and [GridView](#) (and other standard [ItemsControl](#)-derived controls) perform UI virtualization for you. When items are close to being scrolled into view (a few pages away), the framework generates the UI for the items and caches them. When it's unlikely that the items will be shown again, the framework re-claims the memory.

If you provide a custom items panel template (see [ItemsPanel](#)) then make sure you use a virtualizing panel such as [ItemsWrapGrid](#) or [ItemsStackPanel](#). If you use [VariableSizeWrapGrid](#), [WrapGrid](#), or [StackPanel](#), then you will not get virtualization. Additionally, the following [ListView](#) events are raised only when using an

[ItemsWrapGrid](#) or an [ItemsStackPanel](#): [ChoosingGroupHeaderContainer](#), [ChoosingItemContainer](#), and [ContainerContentChanging](#).

The concept of a viewport is critical to UI virtualization because the framework must create the elements that are likely to be shown. In general, the viewport of an [ItemsControl](#) is the extent of the logical control. For example, the viewport of a [ListView](#) is the width and height of the [ListView](#) element. Some panels allow child elements unlimited space, examples being [ScrollViewer](#) and a [Grid](#), with auto-sized rows or columns. When a virtualized [ItemsControl](#) is placed in a panel like that, it takes enough room to display all of its items, which defeats virtualization. Restore virtualization by setting a width and height on the [ItemsControl](#).

Element reduction per item

Keep the number of UI elements used to render your items to a reasonable minimum.

When an items control is first shown, all the elements needed to render a viewport full of items are created. Also, as items approach the viewport, the framework updates the UI elements in cached item templates with the bound data objects. Minimizing the complexity of the markup inside templates pays off in memory and in time spent on the UI thread, improving responsiveness especially while panning/scrolling. The templates in question are the item template (see [ItemTemplate](#)) and the control template of a [ListViewItem](#) or a [GridViewItem](#) (the item control template, or [ItemContainerStyle](#)). The benefit of even a small reduction in element count is multiplied by the number of items displayed.

For examples of element reduction, see [Optimize your XAML markup](#).

The default control templates for [ListViewItem](#) and [GridViewItem](#) contain a [ListViewItemPresenter](#) element. This presenter is a single optimized element that displays complex visuals for focus, selection, and other visual states. If you already have custom item control templates ([ItemContainerStyle](#)), or if in future you edit a copy of an item control template, then we recommend you use a [ListViewItemPresenter](#) because that element will give you optimum balance between performance and customizability in the majority of cases. You customize the presenter by setting properties on it. For example, here's markup that removes the check mark that appears by default when an item is selected, and changes the background color of the selected item to orange.

XML

```
...
<ListView>
    ...
```

```

        <ListView.ItemContainerStyle>
            <Style TargetType="ListViewItem">
                <Setter Property="Template">
                    <Setter.Value>
                        <ControlTemplate TargetType="ListViewItem">
                            <ListViewItemPresenter
                                SelectionCheckMarkVisualEnabled="False" SelectedBackground="Orange"/>
                        </ControlTemplate>
                    </Setter.Value>
                </Setter>
            </Style>
        </ListView.ItemContainerStyle>
    </ListView>
<!-- ... -->

```

There are about 25 properties with self-describing names similar to **SelectionCheckMarkVisualEnabled** and **SelectedBackground**. Should the presenter types prove not to be customizable enough for your use case, you can edit a copy of the `ListViewItemExpanded` or `GridViewItemExpanded` control template instead. These can be found in `\Program Files (x86)\Windows Kits\10\DesignTime\CommonConfiguration\Neutral\UAP\<version>\Generic\generic.xaml`. Be aware that using these templates means trading some performance for the increase in customization.

Update ListView and GridView items progressively

If you're using data virtualization then you can keep **ListView** and **GridView** responsiveness high by configuring the control to render temporary UI elements for the items still being (down)loaded. The temporary elements are then progressively replaced with actual UI as data loads.

Also—no matter where you're loading data from (local disk, network, or cloud)—a user can pan/scroll a **ListView** or **GridView** so rapidly that it's not possible to render each item with full fidelity while preserving smooth panning/scrolling. To preserve smooth panning/scrolling you can choose to render an item in multiple phases in addition to using placeholders.

An example of these techniques is often seen in photo-viewing apps: even though not all of the images have been loaded and displayed, the user can still pan/scroll and interact with the collection. Or, for a "movie" item, you could show the title in the first phase, the rating in the second phase, and an image of the poster in the third phase. The user sees the most important data about each item as early as possible, and that

means they're able to take action at once. Then the less important info is filled-in as time allows. Here are the platform features you can use to implement these techniques.

Placeholders

The temporary placeholder visuals feature is on by default, and it's controlled with the [ShowsScrollingPlaceholders](#) property. During fast panning/scrolling, this feature gives the user a visual hint that there are more items yet to fully display while also preserving smoothness. If you use one of the techniques below then you can set [ShowsScrollingPlaceholders](#) to false if you prefer not to have the system render placeholders.

Progressive data template updates using x:Phase

Here's how to use the [x:Phase attribute](#) with [{x:Bind}](#) bindings to implement progressive data template updates.

1. Here's what the binding source looks like (this is the data source that we'll bind to).

```
C#

namespace LotsOfItems
{
    public class ExampleItem
    {
        public string Title { get; set; }
        public string Subtitle { get; set; }
        public string Description { get; set; }
    }

    public class ExampleItemViewModel
    {
        private ObservableCollection<ExampleItem> exampleItems = new
ObservableCollection<ExampleItem>();
        public ObservableCollection<ExampleItem> ExampleItems { get {
return this.exampleItems; } }

        public ExampleItemViewModel()
        {
            for (int i = 1; i < 150000; i++)
            {
                this.exampleItems.Add(new ExampleItem(){
                    Title = "Title: " + i.ToString(),
                    Subtitle = "Sub: " + i.ToString(),
                    Description = "Desc: " + i.ToString()
                });
            }
        }
    }
}
```

```
}  
}
```

2. Here's the markup that `DeferMainPage.xaml` contains. The grid view contains an item template with elements bound to the **Title**, **Subtitle**, and **Description** properties of the **MyItem** class. Note that **x:Phase** defaults to 0. Here, items will be initially rendered with just the title visible. Then the subtitle element will be data bound and made visible for all the items and so on until all the phases have been processed.

XML

```
<Page  
  x:Class="LotsOfItems.DeferMainPage"  
  xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
  xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
  xmlns:d="http://schemas.microsoft.com/expression/blend/2008"  
  xmlns:mc="http://schemas.openxmlformats.org/markup-  
compatibility/2006"  
  xmlns:lotsOfItems="using:LotsOfItems"  
  mc:Ignorable="d">  
  
  <Grid Background="{ThemeResource  
ApplicationPageBackgroundThemeBrush}">  
    <GridView ItemsSource="{x:Bind ViewModel.ExampleItems}">  
      <GridView.ItemTemplate>  
        <DataTemplate x:DataType="lotsOfItems:ExampleItem">  
          <StackPanel Height="100" Width="100"  
Background="OrangeRed">  
            <TextBlock Text="{x:Bind Title}"/>  
            <TextBlock Text="{x:Bind Subtitle}"  
x:Phase="1"/>  
            <TextBlock Text="{x:Bind Description}"  
x:Phase="2"/>  
          </StackPanel>  
        </DataTemplate>  
      </GridView.ItemTemplate>  
    </GridView>  
  </Grid>  
</Page>
```

3. If you run the app now and pan/scroll quickly through the grid view then you'll notice that as each new item appears on the screen, at first it is rendered as a dark gray rectangle (thanks to the [ShowsScrollingPlaceholders](#) property defaulting to `true`), then the title appears, followed by subtitle, followed by description.

Progressive data template updates using `ContainerContentChanging`

The general strategy for the **ContainerContentChanging** event is to use **Opacity** to hide elements that don't need to be immediately visible. When elements are recycled, they will retain their old values so we want to hide those elements until we've updated those values from the new data item. We use the **Phase** property on the event arguments to determine which elements to update and show. If additional phases are needed, we register a callback.

1. We'll use the same binding source as for **x:Phase**.
2. Here's the markup that `MainPage.xaml` contains. The grid view declares a handler to its **ContainerContentChanging** event, and it contains an item template with elements used to display the **Title**, **Subtitle**, and **Description** properties of the **MyItem** class. To get the maximum performance benefits of using **ContainerContentChanging**, we don't use bindings in the markup but we instead assign values programmatically. The exception here is the element displaying the title, which we consider to be in phase 0.

XML

```
<Page
  x:Class="LotsOfItems.MainPage"
  xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
  xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
  xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
  xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
  xmlns:lotsOfItems="using:LotsOfItems"
  mc:Ignorable="d">

  <Grid Background="{ThemeResource
ApplicationPageBackgroundThemeBrush}">
    <GridView ItemsSource="{x:Bind ViewModel.ExampleItems}"
ContainerContentChanging="GridView_ContainerContentChanging">
      <GridView.ItemTemplate>
        <DataTemplate x:DataType="lotsOfItems:ExampleItem">
          <StackPanel Height="100" Width="100"
Background="OrangeRed">
            <TextBlock Text="{x:Bind Title}"/>
            <TextBlock Opacity="0"/>
            <TextBlock Opacity="0"/>
          </StackPanel>
        </DataTemplate>
      </GridView.ItemTemplate>
    </GridView>
  </Grid>
</Page>
```

3. Lastly, here's the implementation of the [ContainerContentChanging](#) event handler. This code also shows how we add a property of type **RecordingViewModel** to **MainPage** to expose the binding source class from the class that represents our page of markup. As long as you don't have any [{Binding}](#) bindings in your data template, then mark the event arguments object as handled in the first phase of the handler to hint to the item that it needn't set a data context.

C#

```
namespace LotsOfItems
{
    /// <summary>
    /// An empty page that can be used on its own or navigated to
    /// within a Frame.
    /// </summary>
    public sealed partial class MainPage : Page
    {
        public MainPage()
        {
            this.InitializeComponent();
            this.ViewModel = new ExampleItemViewModel();
        }

        public ExampleItemViewModel ViewModel { get; set; }

        // Display each item incrementally to improve performance.
        private void GridView_ContainerContentChanging(ListViewBase
sender, ContainerContentChangingEventArgs args)
        {
            if (args.Phase != 0)
            {
                throw new System.Exception("We should be in phase 0,
but we are not.");
            }

            // It's phase 0, so this item's title will already be bound
            and displayed.

            args.RegisterUpdateCallback(this.ShowSubtitle);

            args.Handled = true;
        }

        private void ShowSubtitle(ListViewBase sender,
ContainerContentChangingEventArgs args)
        {
            if (args.Phase != 1)
            {
                throw new System.Exception("We should be in phase 1,
but we are not.");
            }
        }
    }
}
```

```

        // It's phase 1, so show this item's subtitle.
        var templateRoot = args.ItemContainer.ContentTemplateRoot
as StackPanel;
        var textBlock = templateRoot.Children[1] as TextBlock;
        textBlock.Text = (args.Item as ExampleItem).Subtitle;
        textBlock.Opacity = 1;

        args.RegisterUpdateCallback(this.ShowDescription);
    }

    private void ShowDescription(ListViewBase sender,
ContainerContentChangingEventArgs args)
    {
        if (args.Phase != 2)
        {
            throw new System.Exception("We should be in phase 2,
but we are not.");
        }

        // It's phase 2, so show this item's description.
        var templateRoot = args.ItemContainer.ContentTemplateRoot
as StackPanel;
        var textBlock = templateRoot.Children[2] as TextBlock;
        textBlock.Text = (args.Item as ExampleItem).Description;
        textBlock.Opacity = 1;
    }
}
}

```

4. If you run the app now and pan/scroll quickly through the grid view then you'll see the same behavior as for `x:Phase`.

Container-recycling with heterogeneous collections

In some applications, you need to have different UI for different types of item within a collection. This can create a situation where it is impossible for virtualizing panels to reuse/recycle the visual elements used to display the items. Recreating the visual elements for an item during panning undoes many of the performance wins provided by virtualization. However, a little planning can allow virtualizing panels to reuse the elements. Developers have a couple of options depending on their scenario: the [ChoosingItemContainer](#) event, or an item template selector. The [ChoosingItemContainer](#) approach has better performance.

The ChoosingItemContainer event

[ChoosingItemContainer](#) is an event that allows you to provide an item ([ListViewItem/GridViewItem](#)) to the [ListView/GridView](#) whenever a new item is needed during start-up or recycling. You can create a container based on the type of data item the container will display (shown in the example below). **ChoosingItemContainer** is the higher-performing way to use different data templates for different items. Container caching is something that can be achieved using **ChoosingItemContainer**. For example, if you have five different templates, with one template occurring an order of magnitude more often than the others, then **ChoosingItemContainer** allows you not only to create items at the ratios needed but also to keep an appropriate number of elements cached and available for recycling. [ChoosingGroupHeaderContainer](#) provides the same functionality for group headers.

C#

```
// Example shows how to use ChoosingItemContainer to return the correct
// DataTemplate when one is available. This example shows how to return
// different
// data templates based on the type of FileItem. Available ListViewItem's are
// kept
// in two separate lists based on the type of DataTemplate needed.
private void ListView_ChoosingItemContainer
    (ListViewBase sender, ChoosingItemContainerEventArgs args)
{
    // Determines type of FileItem from the item passed in.
    bool special = args.Item is DifferentFileItem;

    // Uses the Tag property to keep track of whether a particular
    ListViewItem's
    // datatemplate should be a simple or a special one.
    string tag = special ? "specialFiles" : "simpleFiles";

    // Based on the type of datatemplate needed return the correct list of
    // ListViewItems, this could have also been handled with a hash table.
    These
    // two lists are being used to keep track of ItemContainers that can be
    reused.
    List<UIElement> relevantStorage = special ? specialFileItemTrees :
    simpleFileItemTrees;

    // args.ItemContainer is used to indicate whether the ListView is
    proposing an
    // ItemContainer (ListViewItem) to use. If args.Itemcontainer, then
    there was a
    // recycled ItemContainer available to be reused.
    if (args.ItemContainer != null)
    {
        // The Tag is being used to determine whether this is a special file
        or
        // a simple file.
        if (args.ItemContainer.Tag.Equals(tag))
        {
```

```

        // Great: the system suggested a container that is actually
going to    // work well.
    }
    else
    {
        // the ItemContainer's datatemplate does not match the needed
        // datatemplate.
        args.ItemContainer = null;
    }
}

if (args.ItemContainer == null)
{
    // see if we can fetch from the correct list.
    if (relevantStorage.Count > 0)
    {
        args.ItemContainer = relevantStorage[0] as SelectorItem;
    }
    else
    {
        // there aren't any (recycled) ItemContainers available. So a
new one    // needs to be created.
        ListViewItem item = new ListViewItem();
        item.ContentTemplate = this.Resources[tag] as DataTemplate;
        item.Tag = tag;
        args.ItemContainer = item;
    }
}
}

```

Item template selector

An item template selector ([DataTemplateSelector](#)) allows an app to return a different item template at runtime based on the type of the data item that will be displayed. This makes development more productive, but it makes UI virtualization more difficult because not every item template can be reused for every data item.

When recycling an item ([ListViewItem](#)/[GridViewItem](#)), the framework must decide whether the items that are available for use in the recycle queue (the recycle queue is a cache of items that are not currently being used to display data) have an item template that will match the one desired by the current data item. If there are no items in the recycle queue with the appropriate item template then a new item is created, and the appropriate item template is instantiated for it. If, on other hand, the recycle queue contains an item with the appropriate item template then that item is removed from the recycle queue and is used for the current data item. An item template selector works in situations where only a small number of item templates are used and there is a flat distribution throughout the collection of items that use different item templates.

When there is an uneven distribution of items that use different item templates then new item templates will likely need to be created during panning, and this negates many of the gains provided by virtualization. Additionally, an item template selector only considers five possible candidates when evaluating whether a particular container can be reused for the current data item. So you should carefully consider whether your data is appropriate for use with an item template selector before using one in your app. If your collection is mostly homogeneous then the selector is returning the same type most (possibly all) of the time. Just be aware of the price you're paying for the rare exceptions to that homogeneity, and consider whether using [ChoosingItemContainer](#) (or two items controls) is preferable.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

ListView and GridView data virtualization

Article • 10/20/2022

Note For more details, see the [//build/ session Dramatically Increase Performance when Users Interact with Large Amounts of Data in GridView and ListView](#).

Improve [ListView](#) and [GridView](#) performance and startup time through data virtualization. For UI virtualization, element reduction, and progressive updating of items, see [ListView and GridView UI optimization](#).

A method of data virtualization is needed for a data set that is so large that it cannot or should not all be stored in memory at one time. You load an initial portion into memory (from local disk, network, or cloud) and apply UI virtualization to this partial data set. You can later load data incrementally, or from arbitrary points in the master data set (random access), on demand. Whether data virtualization is appropriate for you depends on many factors.

- The size of your data set
- The size of each item
- The source of the data set (local disk, network, or cloud)
- The overall memory consumption of your app

Note Be aware that a feature is enabled by default for ListView and GridView that displays temporary placeholder visuals while the user is panning/scrolling quickly. As data is loaded, these placeholder visuals are replaced with your item template. You can turn the feature off by setting [ListViewBase.ShowsScrollingPlaceholders](#) to false, but if you do so then we recommend that you use the `x:Phase` attribute to progressively render the elements in your item template. See [Update ListView and GridView items progressively](#).

Here are more details about the incremental and random-access data virtualization techniques.

Incremental data virtualization

Incremental data virtualization loads data sequentially. A [ListView](#) that uses incremental data virtualization may be used to view a collection of a million items, but only 50 items are loaded initially. As the user pans/scrolls, the next 50 are loaded. As items are loaded,

the scroll bar's thumb decreases in size. For this type of data virtualization you write a data source class that implements these interfaces.

- [IList](#)
- [INotifyCollectionChanged](#) (C#/VB) or [IObservableVector<T>](#) (C++/CX)
- [ISupportIncrementalLoading](#)

A data source like this is an in-memory list that can be continually extended. The items control will ask for items using the standard [IList](#) indexer and count properties. The count should represent the number of items locally, not the true size of the dataset.

When the items control gets close to the end of the existing data, it will call [ISupportIncrementalLoading.HasMoreItems](#). If you return **true**, then it will call [ISupportIncrementalLoading.LoadMoreItemsAsync](#) passing an advised number of items to load. Depending on where you're loading data from (local disk, network, or cloud), you may choose to load a different number of items than that advised. For example, if your service supports batches of 50 items but the items control only asks for 10 then you can load 50. Load the data from your back end, add it to your list, and raise a change notification via [INotifyCollectionChanged](#) or [IObservableVector<T>](#) so that the items control knows about the new items. Also return a count of the items you actually loaded. If you load fewer items than advised, or the items control has been panned/scrolled even further in the interim, then your data source will be called again for more items and the cycle will continue. You can learn more by downloading the [XAML data binding sample](#) [↗](#) for Windows 8.1 and re-using its source code in your Windows 10 app.

Random access data virtualization

Random access data virtualization allows loading from an arbitrary point in the data set. A [ListView](#) that uses random access data virtualization, used to view a collection of a million items, can load the items 100,000 – 100,050. If the user then moves to the beginning of the list, the control loads items 1 – 50. At all times, the scroll bar's thumb indicates that the **ListView** contains a million items. The position of the scroll bar's thumb is relative to where the visible items are located in the collection's entire data set. This type of data virtualization can significantly reduce the memory requirements and load times for the collection. To enable it you need to write a data source class that fetches data on demand and manages a local cache and implements these interfaces.

- [IList](#)
- [INotifyCollectionChanged](#) (C#/VB) or [IObservableVector<T>](#) (C++/CX)
- (Optionally) [IItemsRangeInfo](#)
- (Optionally) [ISelectionInfo](#)

IItemsRangeInfo provides information on which items the control is actively using. The items control will call this method whenever its view is changing, and will include these two sets of ranges.

- The set of items that are in the viewport.
- A set of non-virtualized items that the control is using that may not be in the viewport.
 - A buffer of items around the viewport that the items control keeps so that touch panning is smooth.
 - The focused item.
 - The first item.

By implementing **IItemsRangeInfo** your data source knows what items need to be fetched and cached, and when to prune from the cache data that is no longer needed. **IItemsRangeInfo** uses **IItemIndexRange** objects to describe a set of items based on their index in the collection. This is so that it doesn't use item pointers, which may not be correct or stable. **IItemsRangeInfo** is designed to be used by only a single instance of an items control because it relies on state information for that items control. If multiple items controls need access to the same data then you will need a separate instance of the data source for each. They can share a common cache, but the logic to purge from the cache will be more complicated.

Here's the basic strategy for your random access data virtualization data source.

- When asked for an item
 - If you have it available in memory, then return it.
 - If you don't have it, then return either null or a placeholder item.
 - Use the request for an item (or the range information from **IItemsRangeInfo**) to know which items are needed, and to fetch data for items from your back end asynchronously. After retrieving the data, raise a change notification via **INotifyCollectionChanged** or **IObservableVector<T>** so that the items control knows about the new item.
- (Optionally) as the items control's viewport changes, identify what items are needed from your data source via your implementation of **IItemsRangeInfo**.

Beyond that, the strategy for when to load data items, how many to load, and which items to keep in memory is up to your application. Some general considerations to keep in mind:

- Make asynchronous requests for data; don't block the UI thread.
- Find the sweet spot in the size of the batches you fetch items in. Prefer chunky to chatty. Not so small that you make too many small requests; not too large that they take too long to retrieve.

- Consider how many requests you want to have pending at the same time. Performing one at a time is easier, but it may be too slow if turnaround time is high.
- Can you cancel requests for data?
- If using a hosted service, is there a cost per transaction?
- What kind of notifications are provided by the service when the results of a query are changed? Will you know if an item is inserted at index 33? If your service supports queries based on a key-plus-offset, that may be better than just using an index.
- How smart do you want to be in pre-fetching items? Are you going to try and track the direction and velocity of scrolling to predict which items are needed?
- How aggressive do you want to be in purging the cache? This is a tradeoff of memory versus experience.

Improve garbage collection performance

Article • 10/20/2022

Universal Windows Platform (UWP) apps written in C# and Visual Basic get automatic memory management from the .NET garbage collector. This section summarizes the behavior and performance best practices for the .NET garbage collector in UWP apps. For more info on how the .NET garbage collector works and tools for debugging and analyzing garbage collector performance, see [Garbage collection](#).

Note Needing to intervene in the default behavior of the garbage collector is strongly indicative of general memory issues with your app. For more info, see [Memory Usage Tool while debugging in Visual Studio 2015](#) . This topic applies to C# and Visual Basic only.

The garbage collector determines when to run by balancing the memory consumption of the managed heap with the amount of work a garbage collection needs to do. One of the ways the garbage collector does this is by dividing the heap into generations and collecting only part of the heap most of the time. There are three generations in the managed heap:

- **Generation 0.** This generation contains newly allocated objects unless they are 85KB or larger, in which case they are part of the large object heap. The large object heap is collected with generation 2 collections. Generation 0 collections are the most frequently occurring type of collection and clean up short-lived objects such as local variables.
- **Generation 1.** This generation contains objects that have survived generation 0 collections. It serves as a buffer between generation 0 and generation 2. Generation 1 collections occur less frequently than generation 0 collections and clean up temporary objects that were active during previous generation 0 collections. A generation 1 collection also collects generation 0.
- **Generation 2.** This generation contains long-lived objects that have survived generation 0 and generation 1 collections. Generation 2 collections are the least frequent and collect the entire managed heap, including the large object heap which contains objects that are 85KB or larger.

You can measure the performance of the garbage collector in 2 aspects: the time it takes to do the garbage collection, and the memory consumption of the managed heap. If you have a small app with a heap size less than 100MB then focus on reducing memory